INFORMATIONSBLATT

# IB01 – Daten speichern mit Prepared Statements

Dynamische Webseiten mit PHP · LS09 | 2BKI2 · Berufskolleg Technische Assistenz · Schuljahr 2026/2027

### 🎯 Darum geht es

Sie speichern Daten mit `INSERT`, übergeben Benutzereingaben ausschließlich über Platzhalter (Prepared Statements) und lesen gefilterte Daten sicher aus.

## 1 | Daten einfügen mit INSERT

```sql
INSERT INTO reservierungen (name, mail, datum, zeit, personen, platz)
VALUES ('Lena Hofmann', 'lena.h@beispiel.de', '2027-07-24', '17:30', 2, 'innen');
```

Spalten, die nicht genannt werden, bekommen ihren Standardwert (`id` zählt automatisch hoch, `erstellt` erhält den aktuellen Zeitpunkt). In PHP kommen die Werte aber aus dem Formular – und genau da lauert die Gefahr.

## 2 | Das Prepared Statement

Ein **Prepared Statement** („vorbereitete Anweisung“) trennt den SQL-Befehl von den Werten. Wo später ein Wert stehen soll, steht ein Fragezeichen. Die Datenbank bekommt zuerst die Vorlage, dann getrennt die Werte – ein Wert kann den Befehl dadurch nie verändern.

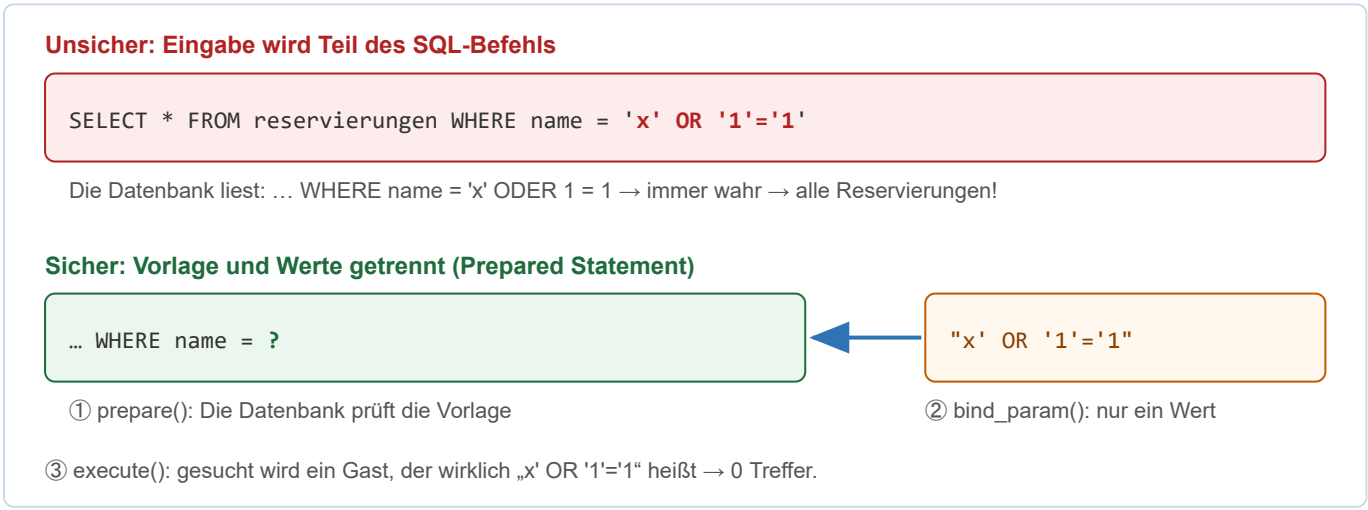



Abbildung 1: Links wird die Eingabe Teil des Befehls, rechts bleibt sie ein Wert.

| Schritt | Code | Bedeutung |
|---|---|---|
| 1 Vorbereiten | `$stmt = $conn->prepare($sql);` | SQL mit `?` an die Datenbank schicken |
| 2 Binden | `$stmt->bind_param("si", $name, $personen);` | Variablen den Fragezeichen zuordnen, mit Typ |
| 3 Ausführen | `$stmt->execute();` | mit den aktuellen Werten ausführen |
| 4 Ergebnis | `$stmt->get_result()` bzw. `$stmt->insert_id` | bei SELECT: Ergebnis wie gewohnt · bei INSERT: neue Nummer |

| Buchstabe | Typ | Beispiel |
|---|---|---|
| `s` | string – Text (auch Datum, Uhrzeit, E-Mail) | `$name`, `$datum` |
| `i` | integer – ganze Zahl | `$personen` |
| `d` | double – Kommazahl | `$preis` |

**INSERT mit Platzhaltern** — beispiel.php

```php
<!DOCTYPE html>
<html lang="de">
<head><meta charset="UTF-8"><title>INSERT</title></head>
<body>

<?php
  require_once "db_verbindung.php";

  $name     = "Jonas Weber";      // kommt sonst aus $_POST
  $personen = 3;

  $stmt = $conn->prepare("INSERT INTO reservierungen (name, mail, datum, zeit, personen, platz)
                          VALUES (?, 'jonas@beispiel.de', '2027-08-01', '16:00', ?, 'innen')");
  $stmt->bind_param("si", $name, $personen);
  $stmt->execute();

  echo "<p>Gespeichert mit der Nummer " . $stmt->insert_id . ".</p>";
?>

</body>
</html>
```

**SO SIEHT ES IM BROWSER AUS**




Abbildung 2: Ein Datensatz wird gespeichert; insert_id liefert die vergebene Nummer.

> ⚠ **REIHENFOLGE UND ANZAHL**
>
> Anzahl der `?`, Anzahl der Typ-Buchstaben und Anzahl der Variablen müssen übereinstimmen, und zwar in derselben Reihenfolge. Sonst bricht PHP mit einem Fehler ab.

## 3 | Sicher lesen: SELECT mit Platzhalter

Auch bei Abfragen gilt: Sobald ein Wert aus einer Eingabe stammt, kommt er über einen Platzhalter. Das Ergebnis holt man mit `get_result()` und durchläuft es wie in LS08.

**Reservierungen eines Tages** tag.php

```php
<!DOCTYPE html>
<html lang="de">
<head><meta charset="UTF-8"><title>SELECT mit Platzhalter</title></head>
<body>

<?php
  require_once "db_verbindung.php";

  $datum = $_GET["datum"];

  $stmt = $conn->prepare("SELECT name, zeit, personen FROM reservierungen WHERE datum = ? ORDER BY zeit");
  $stmt->bind_param("s", $datum);
  $stmt->execute();
  $ergebnis = $stmt->get_result();

  while ($r = $ergebnis->fetch_assoc()) {
    echo "<p>" . $r["zeit"] . " Uhr: " . htmlspecialchars($r["name"]) . " (" . $r["personen"] . " Pers.)</p>";
  }
?>

</body>
</html>
```

**SO SIEHT ES IM BROWSER AUS**

localhost/beispiele/tag.php?datum=2027-07-24

15:00 Uhr: Familie Wagner (5 Pers.)

17:30 Uhr: Lena Hofmann (2 Pers.)

Abbildung 3: Alle Reservierungen am 24.07.2027.

### SUCHEN MIT LIKE

Für „Name enthält …“ gehören die Prozentzeichen in den **Wert**, nicht in die Vorlage: `$muster = "%" . $suche . "%";` und `… WHERE name LIKE ?` mit `bind_param("s", $muster)`.